跨域的概念及解决办法

什么是跨域?

跨域是由浏览器的**同源策略（Same-Origin Policy）**引起的安全限制。

同源的定义

两个 URL 被认为是"同源"需要满足以下三个条件完全相同：

条件	说明
协议	http / https
域名	www.example.com
端口	80 / 443 / 3000 等

跨域示例

假设当前页面是 `http://www.example.com:80/index.html` ：

URL	是否跨域	原因
`http://www.example.com/api`	❌ 同源	协议、域名、端口相同
`https://www.example.com/api`	✅ 跨域	协议不同
`http://api.example.com/api`	✅ 跨域	域名不同
`http://www.example.com:8080/api`	✅ 跨域	端口不同

解决跨域的方法

1. CORS（跨域资源共享）⭐ 最常用

由**后端**设置响应头，允许跨域请求：

```
// Node.js Express 示例
app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*'); // 或指定域名
```

```
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.setHeader('Access-Control-Allow-Credentials', 'true'); // 允许携带 cookie
  next();
});

```

2. 代理服务器 ⭐ 开发环境常用

在你的项目中，可以通过 Vite 配置代理：

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://backend-server.com',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})

```

原理：浏览器请求本地服务器 → 本地服务器转发到目标服务器（服务器之间没有跨域限制）

3. JSONP（仅支持 GET）

利用 `<script>` 标签不受同源策略限制的特点：

```
function jsonp(url, callback) {
  const script = document.createElement('script');
  const callbackName = 'jsonp_' + Date.now();

  window[callbackName] = (data) => {
    callback(data);
    delete window[callbackName];
    document.body.removeChild(script);
```

```
  };

  script.src = `${url}?callback=${callbackName}`;
  document.body.appendChild(script);
}

```

4. Nginx 反向代理

```
server {
    listen 80;
    server_name www.example.com;

    location /api {
        proxy_pass http://backend-server.com;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

5. WebSocket

WebSocket 协议不受同源策略限制，可用于实时通信场景。

6. postMessage

用于不同窗口/iframe 之间的跨域通信：

```
// 发送方
otherWindow.postMessage('Hello', 'https://other-domain.com');

// 接收方
window.addEventListener('message', (event) => {
  if (event.origin === 'https://trusted-domain.com') {
    console.log(event.data);
  }
});
```

```
10
```

推荐方案

场景	推荐方案
开发环境	Vite/Webpack 代理
生产环境	CORS 或 Nginx 反向代理
第三方 API	后端代理转发
实时通信	WebSocket